# Virtual Machines & Virtualization

Hardware / Software, Server, Service & Network Virtualization · Hypervisors · Containers · Cost of Virtualization in the OS

Operating Systems

# Agenda

1. What is virtualization?
2. Hardware and software virtualization
3. Server, service and network virtualization
4. Hypervisors (Type 1 and Type 2)
5. Container virtualization
6. VMs vs containers
7. Cost of virtualization in the OS
8. Summary

# What is Virtualization?

Creating a virtual (software-defined) version of a computing resource — CPU, memory, storage, network or an entire machine — and decoupling it from the physical hardware underneath.

## Key benefits

- Better hardware utilization (often 10–15% → 60–80%)
- Isolation between workloads
- Faster provisioning and cloning
- Snapshots, live migration, easy backup

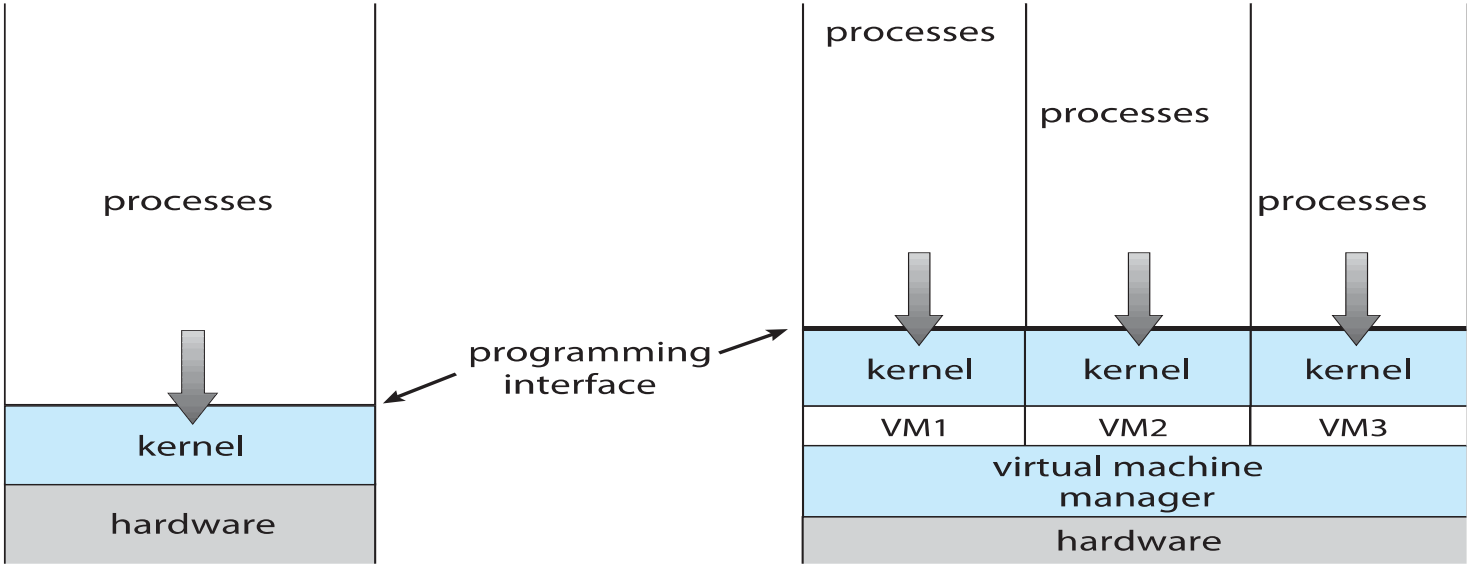
## Core ideas

- Virtual Machine (VM): software computer with its own virtual CPU, RAM, disk and NIC
- Host: physical machine; Guest: the OS inside a VM
- Abstraction layer maps virtual resources to physical ones

# Overview

- Fundamental idea – abstract hardware of a single computer into several different execution environments
  - Similar to layered approach
  - But layer creates virtual system (virtual machine, or VM) on which operation systems or applications can run
- Several components
  - Host – underlying hardware system
  - Virtual machine manager (VMM) or hypervisor – creates and runs virtual machines by providing interface that is ***identical*** to the host
    - (Except in the case of paravirtualization)
  - Guest – process provided with virtual copy of the host
    - Usually an operating system
- Single physical machine can run multiple operating systems concurrently, each in its own virtual machine

# System Models



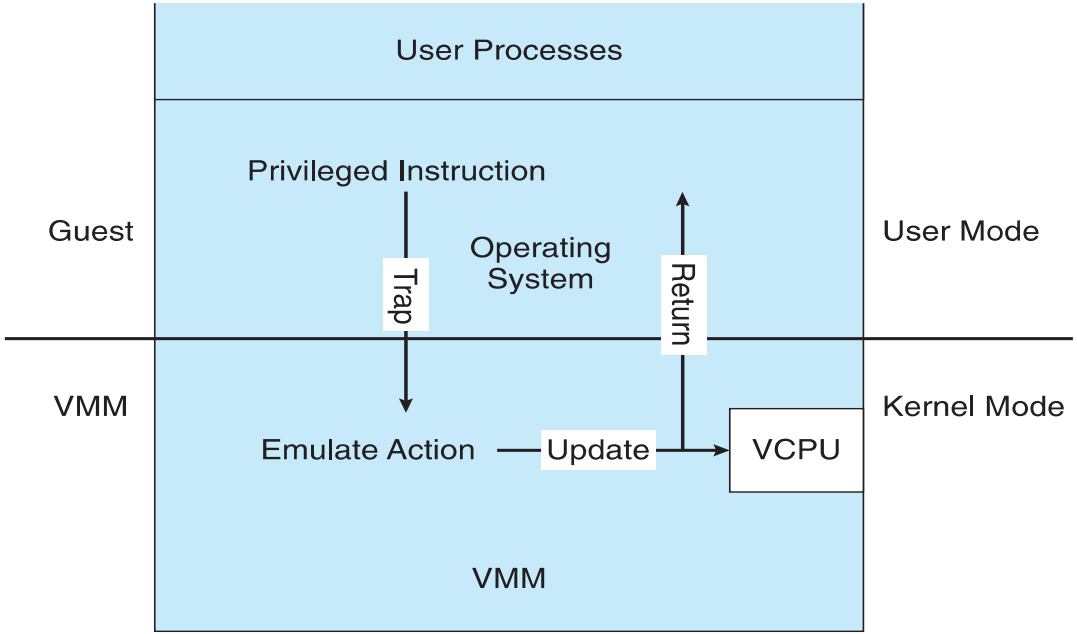

Non-virtual machine

Virtual machine

# Implementation of VMMs

- Vary greatly, with options including:
  - Type 0 hypervisors - Hardware-based solutions that provide support for virtual machine creation and management via firmware
    - IBM LPARs and Oracle LDOMs are examples
  - Type 1 hypervisors - Operating-system-like software built to provide virtualization
    - Including VMware ESX, Joyent SmartOS, and Citrix XenServer
  - Type 1 hypervisors – Also includes general-purpose operating systems that provide standard functions as well as VMM functions
    - Including Microsoft Windows Server with HyperV and RedHat Linux with KVM
  - Type 2 hypervisors - Applications that run on standard operating systems but provide VMM features to guest operating systems
    - Including VMware Workstation and Fusion, Parallels Desktop, and Oracle VirtualBox

# Trap-and-Emulate Virtualization Implementation



# Hardware vs Software Virtualization

## Hardware virtualization

- CPU extensions: Intel VT-x, AMD-V
- Memory: EPT / NPT (nested page tables)
- I/O: Intel VT-d / AMD-Vi (IOMMU), SR-IOV
- Guest runs most instructions directly on the CPU
- Near-native performance; guest OS unmodified

## Software virtualization

- Full virtualization with binary translation (early VMware)
- Paravirtualization: guest OS modified to use hypercalls (Xen PV)
- Emulation (QEMU): complete instruction-by-instruction software model
- Needed where hardware support is absent; slower

# Server Virtualization

Partitioning one physical server into several isolated virtual servers, each running its own OS and applications.

## How it works

- Hypervisor divides CPU, RAM, disk and NIC among VMs
- Each VM boots an independent guest OS
- Resources can be over-committed and rebalanced

## Advantages

- Server consolidation: fewer machines, less power and space
- High availability: live migration (vMotion, Live Migration)
- Disaster recovery via snapshots and replication
- Test/dev environments in minutes

# Service Virtualization

Emulating the behaviour of dependent services (APIs, databases, mainframes, third-party systems) so software can be developed and tested without the real components.

## Why use it

- Dependency is unavailable, costly or rate-limited
- Test failure cases and slow responses on demand
- Parallel teams work without waiting on each other

## Related forms

- Application virtualization: apps run in sandboxes (App-V, Citrix)
- Desktop virtualization (VDI): remote desktops from the data centre
- Storage virtualization: pooled disks (SAN, RAID, LVM)

# Network Virtualization

Combining hardware and software network resources into a single software-based virtual network.

| Technique | Purpose | Examples |
|---|---|---|
| Virtual switch / NIC | Connects VMs inside a host | Open vSwitch, Linux bridge, vSwitch |
| VLAN | Logical segmentation of one physical LAN | IEEE 802.1Q |
| Overlay network | L2 over L3 tunnels across data centres | VXLAN, GRE, Geneve |
| VPN | Secure virtual link over the internet | IPsec, WireGuard |
| SDN / NFV | Central control; network functions as software | OpenFlow, NSX, virtual firewalls |

# Hypervisors: Type 1 vs Type 2

A hypervisor (virtual machine monitor, VMM) creates, runs and manages VMs and arbitrates access to hardware.

| | Type 1 (bare-metal) | Type 2 (hosted) |
|---|---|---|
| Runs on | Directly on hardware | On top of a host OS |
| Performance | High, minimal overhead | Lower: extra host OS layer |
| Typical use | Data centres, cloud | Desktops, development, labs |
| Security surface | Small | Larger (depends on host OS) |
| Examples | VMware ESXi, Xen, Hyper-V, KVM | VirtualBox, VMware Workstation, Parallels |

# How a Hypervisor Virtualizes Resources

## CPU

- Trap-and-emulate for privileged instructions
- VM exits / entries via VT-x / AMD-V
- Scheduler time-slices vCPUs on physical cores

## Memory

- Guest virtual → guest physical → host physical
- Shadow page tables or EPT/NPT
- Ballooning, page sharing, swapping

## I/O

- Emulated devices (slow, compatible)
- Paravirtual drivers: virtio, VMXNET
- Passthrough and SR-IOV (near-native)

# Container Virtualization (OS-level)

Containers share the host kernel and isolate processes instead of virtualizing hardware.

### Kernel building blocks (Linux)

- Namespaces: isolate PIDs, network, mounts, users, hostname
- cgroups: limit CPU, memory, I/O per container
- Union filesystems and layered images (OverlayFS)
- seccomp / capabilities for security

### Ecosystem

- Docker, containerd, Podman, LXC
- Images are portable and start in milliseconds
- Kubernetes orchestrates containers at scale
- Ideal for microservices and CI/CD

# Virtual Machines vs Containers

| Aspect | Virtual Machine | Container |
|---|---|---|
| Isolation level | Hardware-level, own kernel | Process-level, shared kernel |
| Startup time | Seconds to minutes | Milliseconds to seconds |
| Size | GBs (full OS image) | MBs (app + libraries) |
| Density per host | Tens | Hundreds to thousands |
| OS flexibility | Any guest OS | Same kernel family as host |
| Security boundary | Stronger | Weaker; kernel is shared |

*Often combined: containers running inside VMs for both agility and strong isolation.*

# Cost of Virtualization in the Operating System

| Cost | Where it comes from |
| --- | --- |
| CPU overhead | VM exits, trapping privileged instructions, vCPU scheduling (typically ~2–10%) |
| Memory overhead | Per-VM guest OS footprint, page-table duplication, hypervisor metadata |
| I/O overhead | Device emulation, extra copies, virtual switch processing |
| Context switching | World switches between guest and hypervisor, TLB and cache flushes |
| Double scheduling | Guest scheduler runs on top of the host scheduler |
| Noisy neighbours | Shared caches, memory bandwidth and disks cause unpredictable latency |
| Containers | Very low: namespace and cgroup accounting only (~0–3%) |

# Weighing Costs and Benefits

## Costs

- Performance overhead and licensing fees
- Management complexity and VM sprawl
- Single point of failure: host crash affects all VMs
- Hypervisor and side-channel security risks
- Skills and tooling investment

## Savings

- Fewer servers: capex, power, cooling, rack space
- Higher utilization of existing hardware
- Faster deployment and recovery
- Pay-as-you-go cloud elasticity

# Summary

- Virtualization abstracts physical resources into flexible virtual ones.
- Hardware support (VT-x/AMD-V) made full virtualization fast; software techniques fill the gaps.
- Server, service and network virtualization each target a different layer of the stack.
- Type 1 hypervisors suit production; Type 2 suit desktops and labs.
- Containers trade isolation strength for speed and density.
- The OS-level cost is modest but real: CPU, memory, I/O and scheduling overhead versus large consolidation savings.

# Recovery

- Consistency checking – compares data in directory structure with data blocks on disk, and tries to fix inconsistencies
  - Can be slow and sometimes fails
- Use system programs to back up data from disk to another storage device (magnetic tape, other magnetic disk, optical)
- Recover lost file or disk by restoring data from backup

# Log Structured File Systems

- Log structured (or journaling) file systems record each metadata update to the file system as a transaction
- All transactions are written to a log
  - A transaction is considered committed once it is written to the log (sequentially)
  - Sometimes to a separate device or section of disk
  - However, the file system may not yet be updated
- The transactions in the log are asynchronously written to the file system structures
  - When the file system structures are modified, the transaction is removed from the log
- If the file system crashes, all remaining transactions in the log must still be performed
- Faster recovery from crash, removes chance of inconsistency of metadata

# DISTRIBUTED FILE SYSTEMS



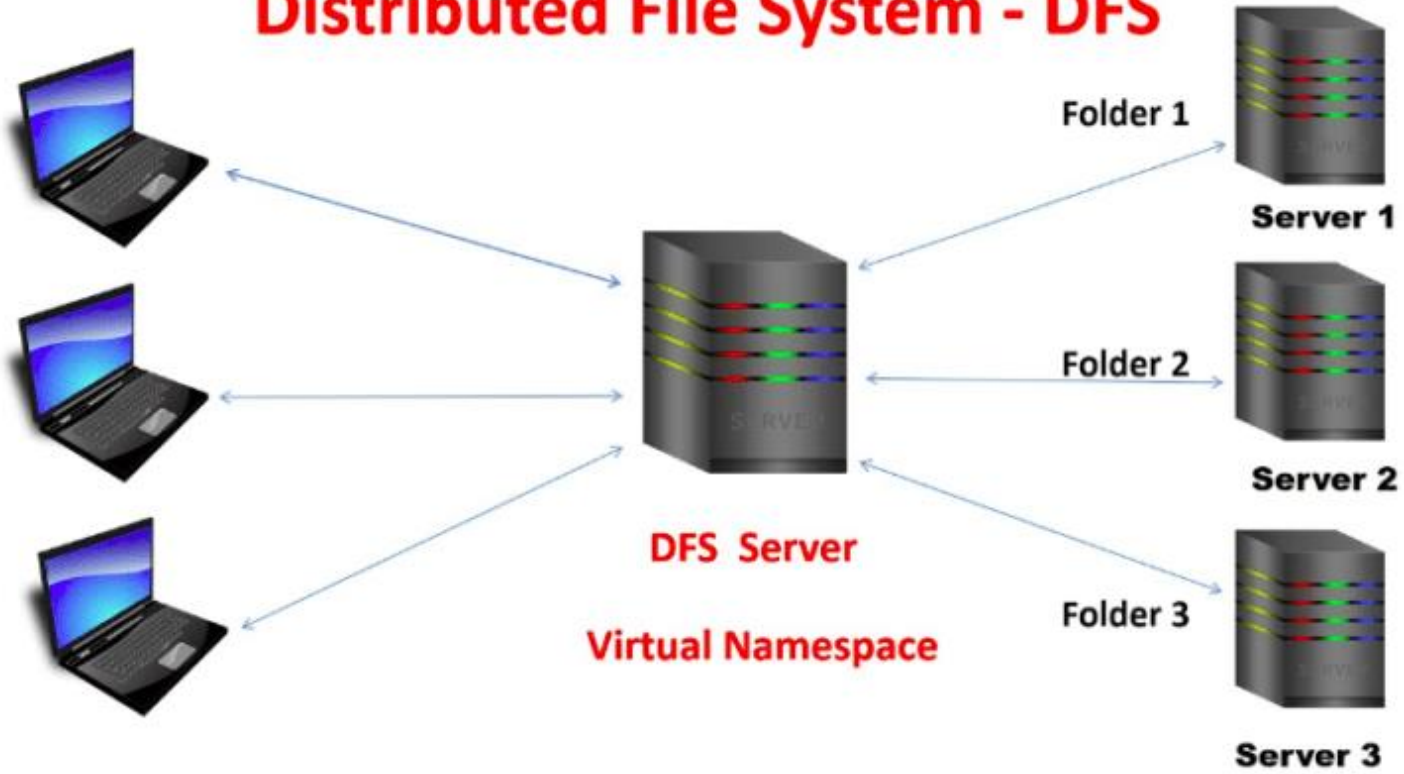
# Distributed File Systems

- DFS is a file system whose clients, servers, and storage devices are dispersed among the machines of a distributed system
- File system that is distributed on multiple file servers or multiple locations
- It allows programs to access or store isolated files as they do with the local ones, allowing programmers to access files from any network or computer.

# DFS Functions

- DFS has two components in its services:
  - **Location Transparency**
  - **Redundancy**
- **Location Transparency**
  - Location transparency is a feature of distributed systems that allows users to access objects without knowing their physical location.
  - This is achieved by using a resource naming scheme that is independent of location.
- **Redundancy**
  - It is achieved via a file replication component.

# Distributed File System - DFS



# Advantages

- It allows the users to access and store the data.
- It helps to improve the access time, network efficiency, and availability of files.
- It provides the transparency of data even if the server of disk files.
- It permits the data to be shared remotely.
- It helps to enhance the ability to change the amount of data and exchange data.

# Disadvantages

- In a DFS, the database connection is complicated.
- In a DFS, database handling is also more complex than in a single-user system.
- If all nodes try to transfer data simultaneously, there is a chance that overloading will happen.
- There is a possibility that messages and data would be missed in the network while moving from one node to another.

# Thank You